

Handoff tecnico V2

Codex / Google AI Studio - Stack oficial React + Vite + NestJS + Prisma + MySQL

Documento	03 - Handoff tecnico V2
Version	2.0 - MySQL oficial
Proyecto	LoanOps RD
Stack oficial	React + Vite + TypeScript NestJS + Prisma MySQL 8+
Autor del proyecto	PholioDev

Este documento reemplaza la version anterior donde pudiera aparecer PostgreSQL, Neon o Supabase como base ideal. La linea oficial de construccion queda cerrada en MySQL 8+ con NestJS y Prisma.

Handoff tecnico V2 para Codex / Google AI Studio

Documento: 03 - Handoff tecnico para desarrollo asistido por IA

Proyecto: LoanOps RD / SaaS para prestamistas

Version: 2.0

Stack oficial: React + Vite + NestJS + Prisma + MySQL

> **Decision oficial de stack:** LoanOps RD se construira con **React + Vite + TypeScript** en frontend, **NestJS + TypeScript** como backend API-first, **Prisma ORM** y **MySQL 8+** como base de datos principal. No se usara PostgreSQL, Neon ni Supabase como base principal en esta fase.

> **Criterio de infraestructura:** se prioriza MySQL porque esta disponible en el entorno actual del proyecto y evita pagar una base de datos adicional. La documentacion queda alineada a esa decision.

1. Proposito del documento

Este documento reemplaza el handoff tecnico anterior y deja la ruta oficial de construccion para Codex, Google AI Studio o cualquier equipo de desarrollo.

Su funcion es evitar improvisaciones en stack, carpetas, rutas, permisos, base de datos, APIs y reglas criticas. La IA debe construir sobre esta guia, no inventar arquitectura paralela.

2. Stack oficial

2.1 Frontend web/admin

- React.
- Vite.
- TypeScript.
- Tailwind CSS.
- Shadcn/UI.
- Lucide Icons.
- React Hook Form.
- Zod para validaciones de frontend.
- TanStack Query para consumo de API y cache.
- Recharts para graficos.

2.2 Backend

- NestJS.
- TypeScript.
- Prisma ORM.
- MySQL 8+.
- JWT + Refresh Tokens.
- Guards y decorators para permisos.
- Servicios modulares por dominio.
- Transacciones Prisma para pagos, prestamos, caja y auditoria.

2.3 Base de datos

- MySQL 8+.
- InnoDB.
- utf8mb4.
- IDs BIGINT UNSIGNED AUTO_INCREMENT.
- Montos con DECIMAL(14, 2).
- Auditoria con campo JSON.
- Indices compuestos con empresa_id.

2.4 App movil futura

- React Native + Expo.
- TypeScript.
- API compartida con NestJS.
- Cola local para modo offline.
- Sincronizacion de pagos, visitas, promesas y rutas.

2.5 PDFs y documentos

- Generacion desde backend.
- Recomendado: Puppeteer para documentos HTML -> PDF o PDFKit si se requiere generacion programatica simple.
- Plantillas configurables por empresa.

2.6 Jobs

Fase inicial sin Redis:

- Cron jobs internos con NestJS Schedule.

Fase Pro/Escala:

- BullMQ + Redis si la infraestructura lo permite.

3. Stack descartado para esta fase

No usar como base principal:

- PostgreSQL.
- Neon.
- Supabase.
- Laravel como backend principal.
- Next.js como backend unico.

Nota: Laravel puede convivir con React/Vite, pero para LoanOps RD se prioriza arquitectura API-first con NestJS por la futura app movil, modulos financieros, auditoria, jobs y separacion frontend/backend.

4. Arquitectura general del repositorio

```
loanops-rd/  
apps/  
web/  
api/  
mobile/  
packages/  
shared/  
ui/  
config/  
docs/  
prisma/  
README.md
```

5. Estructura frontend React + Vite

```
apps/web/src/  
app/  
public/  
admin/  
super-admin/  
components/  
ui/  
layout/  
tables/  
forms/  
cards/  
modals/  
charts/  
pdf/  
features/  
auth/  
dashboard/  
clientes/  
prestamos/  
pagos/  
cobrar-hoy/  
rutas/  
caja/
```

```

reportes/
usuarios/
configuracion/
mi-cuenta/
super-admin/
lib/
api.ts
auth.ts
permissions.ts
formatters.ts
routes.ts
constants.ts
styles/
globals.css
tokens.css

```

5.1 Rutas frontend internas

Ruta	Vista
/dashboard	Dashboard operativo
/cobrar-hoy	Cobros del día
/clientes	Listado de clientes
/clientes/crear	Crear cliente
/clientes/:id	Perfil del cliente
/prestamos	Listado de prestamos
/prestamos/crear	Wizard crear prestamo
/prestamos/:id	Detalle de prestamo
/rutas	Gestion de rutas
/caja	Caja y cierre
/reportes	Reportes
/usuarios	Usuarios
/configuracion	Configuracion
/mi-cuenta	Cuenta del usuario

5.2 Rutas publicas

Ruta	Vista
/	Landing publica
/funciones	Funciones
/como-funciona	Como funciona
/app-movil	App movil
/tutoriales	Centro de tutoriales
/planes	Planes
/contacto	Solicitar demo

Ruta	Vista
/login	Login
/registro	Registro
/recuperar-acceso	Recuperar acceso

6. Estructura backend NestJS

```

apps/api/src/
modules/
auth/
empresas/
sucursales/
usuarios/
clientes/
prestamos/
cuotas/
pagos/
cobrar-hoy/
rutas/
caja/
reportes/
configuracion/
auditoria/
documentos/
notificaciones/
facturacion-saas/
common/
guards/
decorators/
filters/
interceptors/
pipes/
utils/
database/
prisma.service.ts
jobs/
mora.job.ts
promesas-vencidas.job.ts
reportes.job.ts
main.ts

```

7. Variables de entorno

```

NODE_ENV=development
PORT=3001
DATABASE_URL="mysql://user:password@localhost:3306/loanops_rd"
JWT_ACCESS_SECRET="change-me"
JWT_REFRESH_SECRET="change-me"
JWT_ACCESS_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"
APP_TIMEZONE="America/Santo_Domingo"
PDF_STORAGE_PATH="./storage/pdfs"

```

8. Principios tecnicos obligatorios

8.1 Multiempresa primero

Toda consulta operativa debe filtrar por empresa_id desde backend.

```
where: {  
  empresaId: currentUser.empresaId,  
}
```

8.2 Backend valida todo

La interfaz puede ocultar botones, pero el backend decide si una accion es permitida.

8.3 Transacciones financieras

Registrar pago debe ser una transaccion:

1. Crear pago.
2. Actualizar cuota.
3. Actualizar prestamo.
4. Crear movimiento de caja.
5. Generar recibo.
6. Registrar auditoria.

8.4 No borrar datos criticos

- Pagos se anulan, no se eliminan.
- Recibos se anulan, no se eliminan.
- Auditoria no se edita.
- Caja cerrada no se modifica sin reapertura autorizada.

9. Modulos backend y responsabilidad

Modulo	Responsabilidad
auth	Login, refresh, sesiones, recuperacion.
empresas	Tenants y estado de empresas.
sucursales	Segmentacion operativa.
usuarios	Usuarios, roles y permisos.
clientes	CRUD, bloqueo, perfil.
prestamos	Wizard, detalle, reprogramacion, refinanciamiento.
cuotas	Calendario y estados.
pagos	Registro, anulacion, aplicacion financiera.
cobrar-hoy	Listas operativas del dia.

Modulo	Responsabilidad
rutas	Crear, ordenar, iniciar, cerrar rutas.
caja	Movimientos y cierres.
reportes	Metricas, filtros y exportaciones.
configuracion	Reglas por empresa.
auditoria	Eventos sensibles.
documentos	PDFs, recibos y reportes.

10. APIs principales

10.1 Auth

Metodo	Endpoint	Accion
POST	/auth/login	Iniciar sesion
POST	/auth/refresh	Renovar token
POST	/auth/logout	Cerrar sesion
GET	/auth/me	Usuario autenticado
POST	/auth/forgot-password	Solicitar recuperacion
POST	/auth/reset-password	Cambiar contrasena

10.2 Clientes

Metodo	Endpoint	Accion
GET	/clientes	Listar con filtros
POST	/clientes	Crear cliente
GET	/clientes/:id	Perfil
PATCH	/clientes/:id	Editar
POST	/clientes/:id/bloquear	Bloquear
POST	/clientes/:id/desbloquear	Desbloquear

10.3 Prestamos y pagos

Metodo	Endpoint	Accion
GET	/prestamos	Listar prestamos
POST	/prestamos	Crear prestamo
GET	/prestamos/:id	Detalle
GET	/prestamos/:id/cuotas	Cuotas

Metodo	Endpoint	Accion
POST	/pagos	Registrar pago
POST	/pagos/:id/anular	Anular pago
GET	/pagos/:id/recibo	Ver recibo

10.4 Operacion

Metodo	Endpoint	Accion
GET	/cobrar-hoy	Cobros del dia
POST	/visitas	Registrar visita
POST	/promesas	Registrar promesa
GET	/rutas	Listar rutas
POST	/rutas	Crear ruta
POST	/rutas/:id/iniciar	Iniciar ruta
POST	/rutas/:id/cerrar	Cerrar ruta
GET	/caja/resumen	Resumen de caja
POST	/caja/movimientos	Movimiento manual
POST	/caja/cierre	Cierre de caja

11. Prisma - datasource oficial

```
datasource db {
  provider = "mysql"
  url = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}
```

12. Reglas para Codex / Google AI Studio

12.1 Debe hacer

- Usar React + Vite, no Next.js como base principal.
- Usar NestJS en backend.
- Usar Prisma con MySQL.
- Usar TypeScript en todo el stack.
- Aplicar empresa_id en todas las consultas operativas.
- Crear servicios modulares por dominio.
- Usar transacciones en pagos.

- Registrar auditoria en acciones criticas.
- Mantener el UI Kit y el sistema visual ya definido.

12.2 No debe hacer

- No usar PostgreSQL, Neon o Supabase.
- No crear tablas sin empresa_id cuando sean operativas.
- No guardar montos en FLOAT.
- No confiar en calculos del frontend para dinero.
- No borrar pagos aplicados.
- No permitir prestamo a cliente bloqueado.
- No editar rutas cerradas libremente.
- No mezclar datos entre empresas.

13. Orden recomendado de construccion

Fase 1 - Base tecnica

1. Crear monorepo.
2. Configurar React + Vite.
3. Configurar NestJS.
4. Configurar Prisma + MySQL.
5. Crear variables .env.
6. Crear modelos base: empresas, sucursales, usuarios.
7. Crear auth JWT.

Fase 2 - MVP operativo

1. Clientes.
2. Prestamos.
3. Cuotas.
4. Pagos.
5. Recibos.
6. Cobrar Hoy.
7. Caja basica.
8. Dashboard basico.

Fase 3 - Pro

1. Rutas.
2. Visitas.
3. Promesas.
4. Fichas.
5. Reportes.
6. Configuración avanzada.
7. Auditoría completa.

Fase 4 - Escala

1. Super Admin completo.
2. Planes y suscripciones.
3. Facturación SaaS.
4. App móvil.
5. Jobs avanzados.
6. Automatizaciones.
7. IA / scoring.

14. Checklist QA técnico

- DATABASE_URL usa MySQL.
- Prisma provider es mysql.
- Todas las tablas operativas tienen empresa_id.
- Pagos se ejecutan en transacción.
- Pago actualiza cuota, préstamo, caja, recibo y auditoría.
- No hay referencias a PostgreSQL, Neon o Supabase como base principal.
- Cobrador no ve clientes de otra empresa.
- Supervisor no cambia configuración crítica sin permiso.
- Admin Empresa no ve datos de otra empresa.
- Super Admin opera globalmente solo en módulos globales.

15. Conclusion

El stack oficial de LoanOps RD queda cerrado en **React + Vite + TypeScript, NestJS + TypeScript, Prisma ORM y MySQL 8+**. Esta decisión permite construir una plataforma SaaS modular, API-first y compatible con la infraestructura actual sin pagar una base PostgreSQL adicional.